

Lecture 6 -- CSP

- 约束满足问题（CSP）：

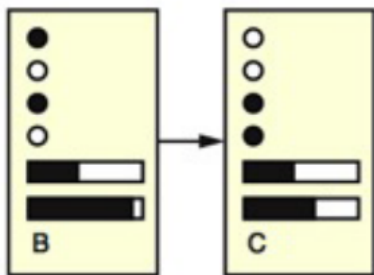
- 是一种搜索问题 Search Problems：

- 找到能达到目标的一连串行动。
 - 行动序列对应搜索空间中的一条路径。
 - 不同路径有不同的代价和深度。
 - 我们使用启发式函数来高效引导搜索。
 - 一个状态是一个“黑盒”（black box），用某种数据结构实现。
 - 回忆“原子表示”（atomic representation）。
 - 目标检测是对状态的函数。



- CSP 问题：

- 一个状态由若干变量 x_i 定义，每个变量的取值来自相应的域 D_i 。回忆“因式化表示”（factored representation）。
 - 变量的种类：
 - 离散变量：
 - 有限域：假设 n 个变量、 d 个取值，则完整赋值的数量为 $O(d^n)$ ，例如地图着色、八皇后问题
 - 无限域（整数、字符串等）：需要使用约束语言，例如作业调度： $T1 + d \leq T2$
 - 连续变量：
 - 在运筹学中常见
 - 含线性或非线性等式的线性规划问题
 - 目标测试是一组约束，规定了针对变量子集的允许取值组合。



- 约束的种类：

- 一元约束：涉及单个变量，例如 $SA \neq \text{green}$

- 二元约束：涉及变量对，例如 $SA \neq WA$
- 全局约束：涉及 3 个或更多变量，例如规定所有变量取不同的值（例如字母数字谜、数独）
- 偏好（软约束）：
 - 示例：红色优于绿色
 - 通常用每个变量赋值的代价表示
 - 约束优化问题
- 更关心目标本身（即满足约束的解），而不是一条动作路径。
- 约束满足问题由三个要素组成：
 - 一组变量 $X = \{X_1, X_2, \dots, X_n\}$
 - 每个变量的取值域集合： $D = \{D_1, D_2, \dots, D_n\}$
 - 一组约束 C ，指定允许的值的组合
- 概念：问题形式化、回溯搜索、弧一致性 等
- 我们把解称为一致的赋值（**consistent assignment**）
- 现实世界的 CSP：
 - 赋值问题，例如：谁教哪门课？
 - 排课问题，例如：哪门课何时何地安排？
 - 硬件配置 / 运输调度 / 工厂排程 / 楼层规划
 - 约束图：

Example: Map coloring

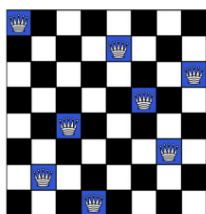


Variables: $X = \{WA, NT, Q, NSW, V, SA, T\}$

Domains: $D_i = \{\text{red, green, blue}\}$

Constraints: adjacent regions must have different colors;
e.g., $WA \neq NT$ or $(WA, NT) \in \{(\text{red, green}), (\text{red, blue}), \text{etc.}\}$

- 二元 CSP：每个约束最多涉及两个变量
 - 约束图：节点表示变量，弧表示约束
- 8 - queen Problem



- 问题形式化 1：

- 每个皇后一个变量, $Q_1, Q_2, \dots, Q_8$
- 每个变量的取值范围为 1 到 64 (对应棋盘上的格子编号)
- 示例解: $Q_1 = 1, Q_2 = 13, Q_3 = 24, \dots, Q_8 = 60$
- 问题形式化 2:
 - 每个皇后一个变量, $Q_1, Q_2, \dots, Q_8$
 - 每个变量的取值范围为 1 到 8 (表示所在行号)
 - 示例解: $Q_1 = 1, Q_2 = 7, Q_3 = 5, \dots, Q_8 = 3$

Example Cryptarithmic 加密算术示例

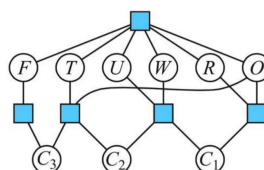
Variables: $X = \{F, T, U, W, R, O, C_1, C_2, C_3\}$

Domain: $D = \{0, 1, 2, \dots, 9\}$

Constraints:

- $\text{Alldiff}(F, T, U, W, R, O)$
- $T \neq 0, F \neq 0$
- $O + O = R + 10 * C_1$
- $C_1 + W + W = U + 10 * C_2$
- $C_2 + T + T = O + 10 * C_3$
- $C_3 = F$

$$\begin{array}{r} T \ W \ O \\ + \ T \ W \ O \\ \hline F \ O \ U \ R \end{array}$$



15

- 求解 CSP: 找到满足所有约束的赋值 (assignment)
 - 状态空间搜索算法: 搜索!
 - CSP 算法:
 - 搜索 search: 从许多可能性中选择新的变量赋值
 - 推理 inference: 约束传播, 利用约束传播影响——减少某变量的取值数量, 从而减少其他变量的合法取值等
 - 作为预处理步骤, 约束传播有时可以在不进行搜索的情况下完全解决问题。
 - 约束传播可以与搜索交织进行。
 - BTS (回溯搜索): 是 CSP 的基本无信息搜索方法。它是一种深度优先搜索 DFS, 具体为:
 - 每次为一个变量赋值 (展开): 赋值是可交换的, 例如 (WA = red, NT = green) 与 (NT = green, WA = red) 等价
 - 及时检查约束: 只考虑与先前赋值不冲突的取值
 - Solving CSPs:
 - 初始状态: 空赋值 {}
 - 状态: 为部分赋值
 - 后继函数: 为一个未赋值的变量分配一个值
 - 目标测试: 当前赋值是完整的并且满足所有约束
 - 示例:

Solving CSPs: Sudoku

All 3x3 boxes, rows, columns, **must contain all digits 1...9.**

Variables: $V = \{A_1, \dots, A_9, B_1, \dots, B_9, \dots, I_1, \dots, I_9\}, |V| = 81.$

Domain: $D = \{1, 2, \dots, 9\}$, the filled squares have a single value.

Constraints: 27 constraints

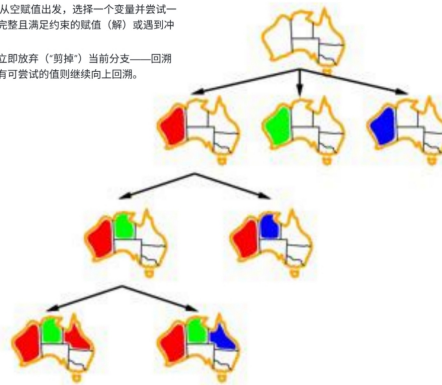
- Alldiff($A_1, A_2, A_3, A_4, A_5, A_6, A_7, A_8, A_9$)
...
- Alldiff($A_1, B_1, C_1, D_1, E_1, F_1, G_1, H_1, I_1$)
...
- Alldiff($A_1, A_2, A_3, B_1, B_2, B_3, C_1, C_2, C_3$)

8		9	5		1	7	3	6
2		7		6	3			
1	6							
				9		4		7
	9		3		7		2	
7		6		8				
							6	3
			9	3		5		2
5	3	2	6		4	8		9

- 裸对、裸三（Naked doubles (triples)）：在某个 3×3 小格中找到两个（或三个）单元格，它们仅剩相同的候选数；则可以从该小格中其它单元格的候选集合中排除这两个（或三个）数。
 - 锁定对（Locked pair）、锁定三（Locked triples）等技巧。
- Improving BTS 启发式回来了！

Backtracking search 回溯搜索

- 基本思想（对应图示的搜索树）：
 - 回溯搜索是对变量逐个赋值的深度优先搜索（DFS）。从空赋值出发，选择一个变量并尝试一个值，进入下一层继续为下一个变量赋值，直到找到完整且满足约束的赋值（解）或遇到冲突。
 - 当一个赋值导致与已有赋值冲突（违反某约束）时，立即放弃（“剪掉”）当前分支——回溯到上一个变量，尝试该变量的下一个可能取值；若没有可尝试的值则继续向上回溯。



Backtracking search

```
function BACKTRACKING_SEARCH(csp) returns a solution, or failure
  return BACKTRACK({}, csp)

function BACKTRACK(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment  如果 assignment 已经是完整赋值(覆盖所有变量), 则返回该 assignment (找到解)
  var = SELECT_UNASSIGNED_VARIABLE(csp)
  for each value in ORDER_DOMAIN_VALUES (var, assignment, csp)
    if value is consistent with assignment then
      add {var = value} to assignment
      result = BACKTRACK(assignment, csp)
      if result ≠ failure then return result
      remove {var = value} from assignment
  return failure
```

Backtracking w/inference

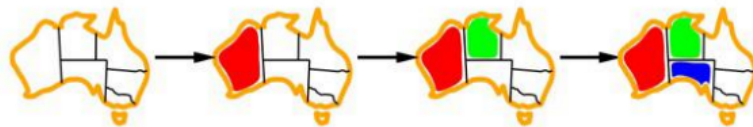
```
function BACKTRACKING-SEARCH(csp) returns a solution or failure
  return BACKTRACK(csp, { })

function BACKTRACK(csp, assignment) returns a solution or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
  for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
    if value is consistent with assignment then
      add { var = value } to assignment
      inferences ← INFERENCE(csp, var, assignment)
      if inferences ≠ failure then
        add inferences to csp
        result ← BACKTRACK(csp, assignment)
        if result ≠ failure then return result
        remove inferences from csp
      remove { var = value } from assignment
  return failure
```

- 应该接下来给哪个变量赋值？

- Minimum Remaining Value (MRV, 最小剩余值启发)**

- 在当前部分赋值状态下，选择那个在其域中“合法取值数目最少”的未赋值变量来优先赋值（即剩余可选值最少的变量）。

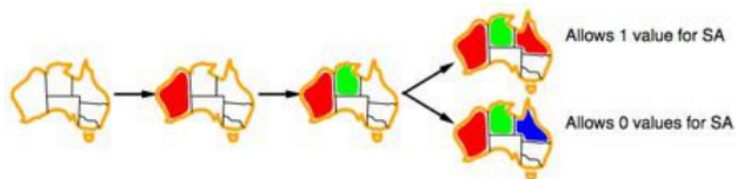


Pick the hardest!

- 应该以何种顺序尝试它的取值？

- Least constraining value (最少约束值)**

- 对于给定变量，选择“最少约束他人”的取值——即在剩余未赋值变量中排除取值最少的那个。

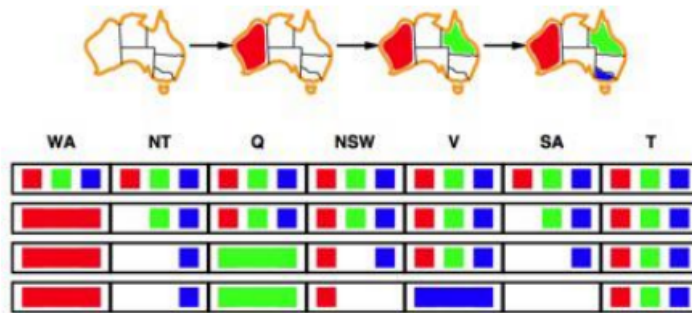


Pick the ones that are likely to work!

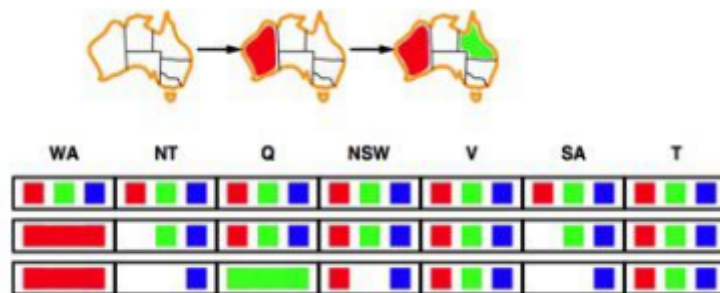
- 我们能否及早检测到不可避免的失败？

- Forward checking (前向检查)**

- 跟踪每个未赋值变量剩余的合法取值。当任一变量没有合法取值时，立即终止（回溯）。



- **Constraint propagation (约束传播) :**
 - 前向检查将已赋值变量的信息传播到未赋值变量。
 - 注意：前向检查并不检查未赋值变量之间的相互作用！
 - 例如这里的 SA 和 NT（它们都被迫为蓝色，但实际上不能同时为蓝色）



- 前向检查能改进回溯搜索，但它对未来的推断范围有限，因此不能检测出所有失败情形。
- 我们使用约束传播，在约束之间进行推理，例如弧一致性（arc consistency）检测。
 - 节点一致性（Node-consistency，针对一元约束）：若变量 X_i 的域 $\text{Domain}(X_i)$ 中的所有值都满足所有一元约束，则称 X_i 为节点一致的。
 - 弧一致性（Arc-consistency，针对二元约束）：若且仅若对 X 的每个值 x ，都存在 Y 的某个值 y ，使得 (x,y) 满足 X 与 Y 之间的约束，则有向弧 $X \rightarrow Y$ 是弧一致的。
 - 例： $\text{Domain}(A) = \{3,4,5\}$, $\text{Domain}(B) = \{3,4,5,6\}$, 约束 $A < B$ 。

Arc consistency

Algorithm that makes a CSP arc-consistent!

```
function AC-3( csp )
  returns False if an inconsistency is found, True otherwise
  inputs: csp, a binary CSP with components  $(X, D, C)$ 
  local variables: queue, a queue of arcs, initially all the arcs in csp
  while queue is not empty do
     $(X_i, X_j) = \text{REMOVE-FIRST}(\text{queue})$ 
    if  $\text{REVISE}(\text{csp}, X_i, X_j)$  then
      if size of  $D_i = 0$  then return False
      for each  $X_k$  in  $X_i.\text{NEIGHBORS} - \{X_j\}$  do
        add  $(X_k, X_i)$  to queue
  return true  // 队列处理完毕且未出现空域，返回 True

function REVISE( csp,  $X_i, X_j$  )
  returns True iff we revise the domain of  $X_i$ 
  revised = False
  for each  $x$  in  $D_i$  do
    if no value  $y$  in  $D_j$  allows  $(x, y)$  to satisfy the constraint between  $X_i$  and  $X_j$  then
      delete  $x$  from  $D_i$ 
      revised = True
  return revised
```

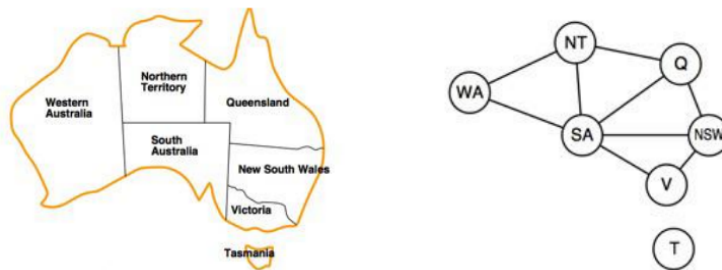
- AC-3 的目标是把一个二元 CSP 变为“弧一致”的形式——也就是对每条有向弧 $(X_i \rightarrow X_j)$ 保证： X_i 域中的每个值 x 都至少有一个在 X_j 域中的支持值 y ，使得 (x,y) 满足 X_i 与 X_j 之间的约束。
- 如果在某次处理后某变量域被删空，则说明该 CSP 无解（返回失败）；否则在所有弧都满足弧一致后返回成功（域被剪枝以便后续搜索更快）。

- 总体复杂度为 $O(n^2 d^3)$

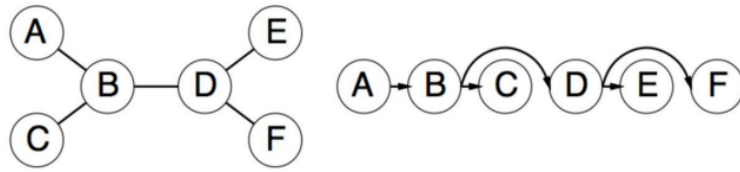
- 设 n 为变量个数, d 为域大小。
- 若每个节点 (变量) 都与其他变量相连, 则有 $n \cdot (n-1)$ 条弧 (约束) $\rightarrow O(n^2)$ 。
- 每条弧最多可被插入队列 d 次 $\rightarrow O(d)$ 。
- 检查一条弧的一致性代价为 $O(d^2)$ (对 x_i 的每个值检查 x_j 的可能支持)。
- 路径一致性与 k -一致性 (针对 n 元约束): 将弧一致性从二元推广到多元约束 (在更大范围内保证一致性)。
- 注意 (Note): 理论上可以把任意 n 元约束转换成二元约束; 许多 CSP 求解器都以二元约束为工作对象。

- Problem structure 问题结构:

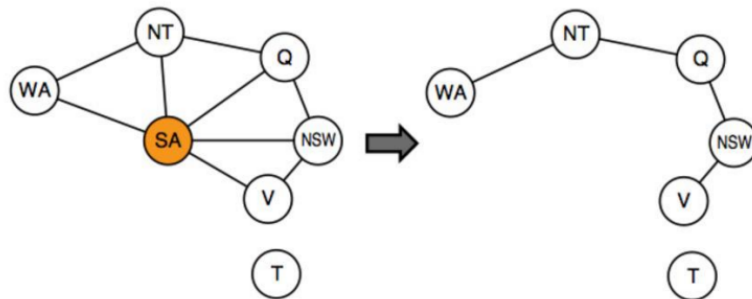
- 利用图结构加速搜索。
- 示例: 塔斯马尼亚 (Tasmania) 是一个独立的子问题。



- 识别约束图的连通分量 (connected components)。
- 在独立的子问题上分别求解。
- 复杂度:
 - 设 d 为域大小, n 为变量数量。
 - BTS (回溯搜索) 的时间复杂度为 $O(d^n)$ 。
 - 假设把问题分解为若干子问题, 每个子问题有 c 个变量。
 - 有 n/c 个子问题。
 - 每个子问题 (c 个变量) 需要 $O(d^c)$ 时间。
 - 所有子问题合计在最坏情况下需要 $O((n/c) \cdot d^c)$ 。
- 将问题分解为独立子问题并不总是可行。我们能否利用其它图结构?
 - 可以, 如果图是树结构或近似树结构。
 - 如果任意两个变量之间仅由一条路径相连, 则该图称为树 tree。
 - 思路: 使用 DAC (有向弧一致性, Directed Arc Consistency)。
 - 若在某一排序 $x_1, x_2, \dots, x_n$ 下每个 x_i 对于所有 $j > i$ 与 x_j 都是弧一致的, 则称该 CSP 在该排序下为有向弧一致 (directed arc-consistent)。



- 首先选择一个变量作为根节点。
- 进行**拓扑排序**：选择变量的一个顺序，使得树中每个变量都在其父节点之后出现。
- 对于 n 个节点，有 $n-1$ 条边。
- 使树具有有向弧一致性需要 $O(n)$ 时间。
- 每次一致性检查最多耗费 $O(d^2)$ （比较两个变量的 d 个可能取值）。
- 因此该 CSP 可在 $O(n d^2)$ 时间内被求解。
- Nearly tree-structured CSPs



- 给某个变量或一组变量赋值，并修剪所有邻居的域。
- 这将把约束图变为一棵树

• 总结：

- CSP 是一种特殊的搜索问题：
 - 状态由固定变量集合的取值定义
 - 目标测试由变量取值上的约束定义
- 回溯搜索 = 每个节点赋值一个变量的深度优先搜索。
- 变量选择和取值顺序的启发式有帮助。
- 前向检查可防止那些必然导致后续失败的赋值。
- 约束传播（例如弧一致性）是 CSP 中的重要机制。
- 它通过额外的工作来约束取值并检测不一致性。
- 树结构的 CSP 可以在线性时间内求解。
- 进一步探索：如何将局部搜索用于 CSP？
- CSP 的强大之处在于与领域无关——你只需定义问题，然后使用实现 CSP 机制的求解器即可。